

Infraestructura como Código (IaC)

Introducción y selección de herramientas

¿Por qué usar IaC?

Reutilización, automatización y escalabilidad ágil: Automatiza la creación de entornos, puedes desplegar en muchos entornos rápidamente con el mismo código y aumentar recursos fácilmente para adaptarse a picos de demanda.

Seguridad mejorada: Facilita establecer y validar políticas de seguridad mediante pruebas automáticas, evitando configuraciones inseguras.

Consistencia: Elimina errores humanos al mantener uniformidad en desarrollo, pruebas y producción.

Compliance: Alinea el despliegue de infraestructuras con los objetivos de calidad y cumplimiento de la organización.

Eficiencia: Aumenta la productividad permitiendo crear entornos aislados, compartir plantillas seguras y automatizar despliegues.

¿Por qué usar IaC?

Reducción de costes: Ahorra en recursos humanos y materiales al evitar configuraciones manuales y mejorar los tests.

Confiabilidad: Las configuraciones se aplican de forma precisa y repetible, reduciendo errores comunes en entornos grandes.

Detección de desviaciones: Identifica y corrige cambios no deseados en la infraestructura frente a la configuración original.

Facilita la experimentación: Permite probar ideas rápidamente sin grandes inversiones, escalando solo lo que funciona.

En realidad.. todo esto son ventajas que los entornos de desarrollo de software han estado adquiriendo durante décadas hasta desembocar en CI/CD: lo que permite IaC es incorporar todas esas ventajas preexistentes a la administración y despliegue de sistemas.

Principales herramientas IaC

Es muy habitual el uso de herramientas de IaC comunes para la configuración automatizada de SO y apli



Pero no son un medio adecuado para el aprovisionamiento de infraestructura cloud y servicios en la nube (sí por ejemplo, para flotas de máquinas virtuales concretas dentro de la misma).

Principales herramientas IaC (cloud)

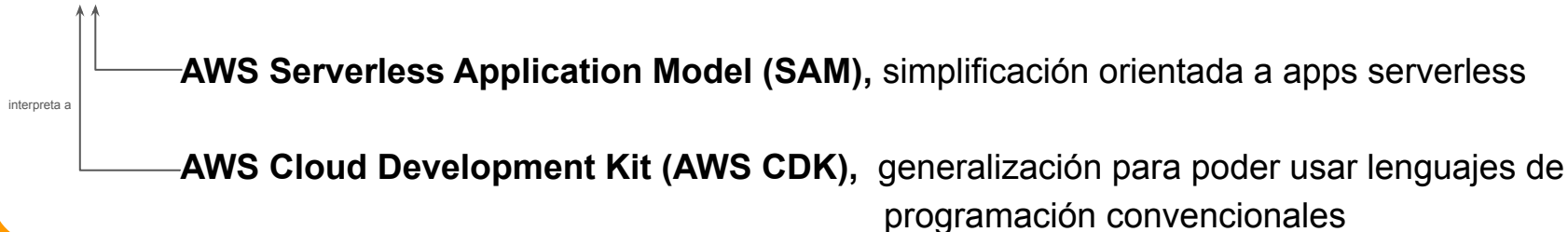
Vamos a explorar **herramientas de IaC** que permiten **aprovisionar recursos en la nube**, especialmente **en AWS**, pero también consideraremos su compatibilidad con **entornos multi-cloud o híbridos**, aspecto clave para organizaciones que buscan flexibilidad.

Cada herramienta será evaluada en función de su **facilidad de uso**, **capacidad de integración** con servicios en la nube, y potencia para **automatizar infraestructuras complejas**. También se tendrán en cuenta aspectos como el **nivel de experiencia en programación** que requieren, no perdamos de vista que estamos trasladando ventajas del desarrollo de software.

Quizá el enfoque que mejor ayude a entender cada herramienta para que esté alineada con las habilidades y objetivos que se persigan sea destacar **en qué escenarios de uso son más útiles o ventajosas** cada una de ellas.

Principales herramientas IaC (cloud)

AWS CloudFormation, IaC de bajo nivel, compleja y fuertemente integrada en AWS



CDKTF, esfuerzo universalizador: “terraformar” CDK para usarlo en otros proveedores cloud

HashiCorp Terraform, herramienta de IaC universal por excelencia y longevidad

Pulumi (y otros), entendido como un *restart* de Terraform a partir de necesidades y situaciones actuales

AWS Cloudformation

Es un servicio IaC nativo que permite definir y provisionar infraestructura en AWS a partir de plantillas, llamadas *stacks*, en JSON o YAML. Con el tiempo ha acumulado importantes ventajas:

- Previsualizar cambios antes de aplicarlos con ***change sets***, manejando mejor errores críticos.
- Si algo falla durante una actualización, puedes hacer ***rollback*** al estado anterior.
- Ideal para despliegues en organizaciones a través de varias cuentas y regiones (con ***stack sets***).
- Visión completa: Permite importar recursos preexistentes, gestiona el estado de la infraestructura, detecta desviaciones (***drifting***) y es la base para disparar mitigaciones y correcciones automáticas (con otros frameworks como Config+EventBridge/SSM).
- Herramienta gratuita, sólo se paga por lo que despliegas.

AWS Cloudformation

No es menos cierto que **tiene una curva de aprendizaje elevada**: definir bien los comportamientos desde ficheros JSON/YAML a bajo nivel es difícil y, su gran ventaja, la profunda integración con AWS, es también una desventaja en términos de dependencia de un sólo proveedor: **fuera de AWS no sirve**.

¿Cuándo usarlo?

- Cuando se necesita **despliegues altamente consistentes y repetibles**, con plantillas de infraestructura complejas con muchos niveles y recursos muy concretos.
- Si la infraestructura, aparte de ser compleja, **está totalmente ubicada -o casi- en AWS**.
- Existen **fuertes requerimientos de compliance** para auditorías, calidad y estándares, que van a requerir excepcionales y detalladas funcionalidades de auditoría.

AWS Serverless Application Model (SAM)

Es un *framework* específico para desarrollo de aplicaciones serverless, que extiende la funcionalidad de CloudFormation para simplificar la sintaxis y **orientarla a herramientas serverless** más específicas como *API Gateway*, *Lambda* o *Step Functions*. **Al final, se traduce a CloudFormation.**

Como **ventajas**, simplificación aparte, destaca el **poder hacer tests en local** y la **profunda integración** con servicios serverless de AWS.

Como **inconvenientes**, tenemos que seguir sabiendo CloudFormation hasta cierto punto, **depende totalmente de ello, arrastrando sus desventajas.**

¿Cuándo usarlo?

Ideal para desarrollos serverless centrados en AWS (que es el principal proveedor), donde las capacidades de **depuración y testeo en local** son vitales para un rápido prototipado y evitar sorpresas.

AWS Cloud Development Kit (AWS CDK)

Es un *framework* que soporta el uso de lenguajes como JavaScript, TypeScript, Python, Java, C#... para el despliegue de infraestructura en AWS. **Al final, se traduce a CloudFormation.**

Es **un nivel de abstracción mayor** con lenguajes de alto nivel, **modularidad y reutilización...** Por tanto, tiene una **curva de aprendizaje más sencilla y facilita la gestión de proyectos grandes.**

En definitiva, es más la **C** porque toma ventaja de los conocimientos de programación e ingeniería del software generales y eso se alinea muy bien con el objetivo de **trasladar las ventajas preexistentes en el desarrollo de software al mundo de la administración de sistemas** que comentamos antes.

Pero no todo son ventajas: los *constructs* se traducen a CloudFormation y **debemos tener conocimientos de base** al respecto y, en todo caso, **al final del día sigues estando limitado a AWS.**

Terraform

Es un *framework* de IaC con **vocación universalizadora**, sólidos principios de **reutilización** y cuyo objetivo principal es ser la **documentación viva** de cómo es la infraestructura y que al mismo tiempo sea un código en archivos que pueden **ejecutarse directamente** para crear o modificar los recursos.

- Puede aplicarse para definir **infraestructura on-prem y en la nube**, para los principales **proveedores** como AWS, Azure o Google Cloud **sin necesidad de agentes específicos**.
- Su **lenguaje de configuración** (HCL) está diseñado para ser **fácilmente entendible tanto por personas como por sistemas**, facilitando la creación de herramientas para procesar o revisar.
- Derivado de lo anterior, **es fácil de aprender y tiene una comunidad fuerte** que le da soporte, con multitud de herramientas, plugins y adaptaciones.
- Soporta **detección de cambios en la infraestructura** para mantener el estado ante modificaciones no deseadas.

Terraform

Lógicamente, también podemos encontrarle una serie de **desventajas**:

- En un mundo dinámico como la nube, **puede ser incompatible con las últimas novedades y por tanto entorpecer la experimentación y la agilidad.**
- **La gestión del estado no es automática** y puede ser compleja o inviable en despliegues de organizaciones a partir de un cierto volumen de cuentas y regiones.
- **No tiene una gestión de secretos** (sobre todo en ficheros de estado), que debe implementarse de manera concreta para cada proveedor, lo que puede llevar a brechas de seguridad
- Desde 2023 ha pasado de licencia *Mozilla Public License* a *Business Source License*. Esto **está minando el ecosistema** (v.g. *OpenTofu*) y desincentivando su adopción para desarrollo de herramientas comerciales de gestión, SaaS, desarrollo de startups, etc.

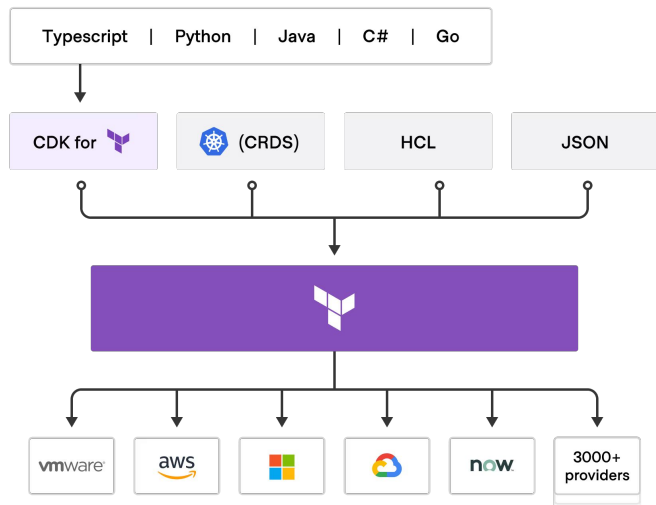
Pulumi

Es una herramienta IaC de código abierto, que **permite el uso de lenguajes de programación de alto nivel** (TypeScript, JavaScript, Python, Go, .NET, Java...) y utiliza un **ecosistema nativo de SDK para interactuar con decenas de proveedores cloud**, ofrecido como librerías y runtimes o bien como servicio alojado.

En definitiva, es como hacer un **restart de CDK** (pero sin estar atado a un proveedor) y **Terraform** (pero de forma nativa orientada a la nube) de cero **sin ningún peaje de antigüedad o compatibilidad**, y orientado en todo momento a definir **un nivel más alto que enmascara la complejidad de la nube**.

Ello deriva en su **principal desventaja**: es un producto nuevo, con **limitada adopción, limitada documentación y limitada comunidad**, donde encontrar respuestas y soporte puede ser difícil, por lo que aunque es una **apuesta valiente con un gran potencial**, el tiempo dirá si va ganando popularidad y fiabilidad.

CDK for Terraform (CDKTF)



En una línea parecida está este *framework*, que es un **esfuerzo universalizador** para utilizar una herramienta fuertemente centrada en AWS, como es **CDK**, y extender su uso a diferentes proveedores mediante **Terraform**, **aunando lo mejor de los dos mundos**.

Así, quien tenga conocimientos de CDK y sea su principal entorno de trabajo y mejor herramienta siendo AWS el principal proveedor cloud, puede tener una ventana abierta a extender su trabajo a otros proveedores **sin acometer Terraform de cero**.

La **principal desventaja** es su **experimentalidad y discreta adopción**: aunque tiene bastante vida y estaba bien mantenida de manera oficial, parece que en 2026 está en retirada.

Conclusiones para elección de herramienta IaC

Como se ha visto, (y esto vale para cualquier tecnología) ninguna herramienta sirve para todo. Aparte de los pros y contras vistos, siempre se pueden hacer recomendaciones para casos específicos:

AWS SAM. Perfecto para aplicaciones *serverless* en AWS (proveedor más común para ello) con pocas dependencias (externas o internas): ofrece la potencia de Cloudformation simplificada y tests en local.

AWS CloudFormation. Para infraestructuras exclusivamente en AWS, ofrece gestión de estado, compliance y auditoría, e integración profunda y agilidad solo dentro de AWS sin más herramientas.

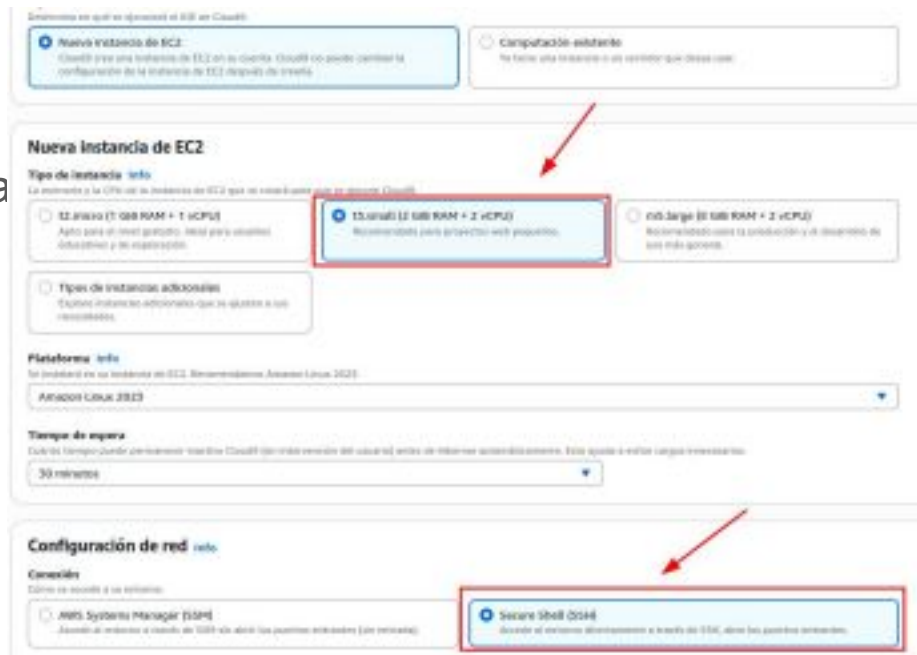
AWS CDK. Es perfecto si la organización se alinea con buenas prácticas de desarrollo de software: equipos, lógica compleja, reutilización... redunda en una administración de infraestructura muy sólida.

Terraform. Cuando se quieren gestionar infraestructuras multi-cloud y on-prem, aún perdiendo agilidad y últimas novedades, pero con una sólida comunidad detrás y opciones de soporte empresarial.

Pulumi. Si se puede asumir cierto nivel de riesgo y hace falta operar en entornos multi-cloud o híbridos. Usa lenguajes de programación y ofrece gran flexibilidad, pero el soporte es difícil y le falta madurez.

Entorno para prácticas

- Utilizaremos un Learner Lab de AWS Academy, que os llegará por invitación de email, el profesor lo indica en la plataforma.
- Dentro del mismo, utilizaremos el servicio **Cloud9**, estableciendo un entorno con máquina **t3.small** (para evitar ir muy justos de recursos) y **acceso por SSH**.



Se recomienda en qué se ejecute el EC2 en Cloud9:

- ☒ Nueva instancia de EC2: Cloud9 crea una instancia de EC2 en su cuenta. Cloud9 no puede cambiar la configuración de la instancia de EC2 después de crearla.
- ☐ Computación existente: No hace una instancia en su cuenta que Cloud9 use.

Nueva instancia de EC2

Tipo de instancia [info](#)

La instancia y la CPU van en instancias de EC2 que se instalan en su cuenta Cloud9.

- ☐ t2.micro (1 GB RAM + 1 vCPU): Apto para el nivel gratuito. Ideal para cargas de trabajo y de desarrollo.
- ☒ t3.small (3 GB RAM + 2 vCPU): Recomendado para proyectos web pequeños.
- ☐ m5.large (8 GB RAM + 2 vCPU): Recomendado para la producción y de desarrollo de los más grandes.

☐ Tipos de instancias adicionales: Elige instancias adicionales que se añaden a las recomendadas.

Plataforma [info](#)

No puedes elegir su instancia de EC2. Recomendaciones Amazon Linux 2023:

Amazon Linux 2023

Tiempo de espera

Elige el tiempo cuando quieras crear tu instancia Cloud9 (se creará cuando el sistema de reservas acepte la instancia). Esto puede tardar algunos minutos.

30 minutos

Configuración de red [info](#)

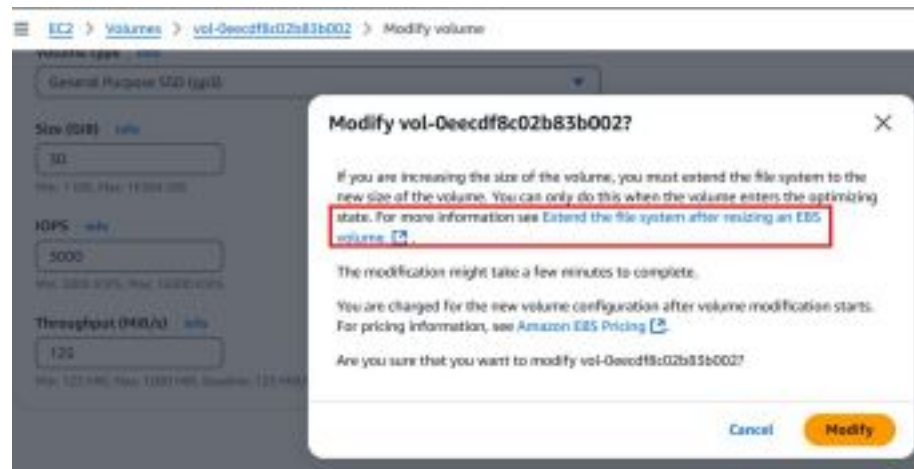
Conexión

Elige la conexión a la instancia:

- ☐ AWS Systems Manager (SSM): Accede al ordenador a través de SSM sin abrir los puertos de red (por internet).
- ☒ Secure Shell (SSH): Accede al ordenador directamente a través de SSH, abre los puertos necesarios.

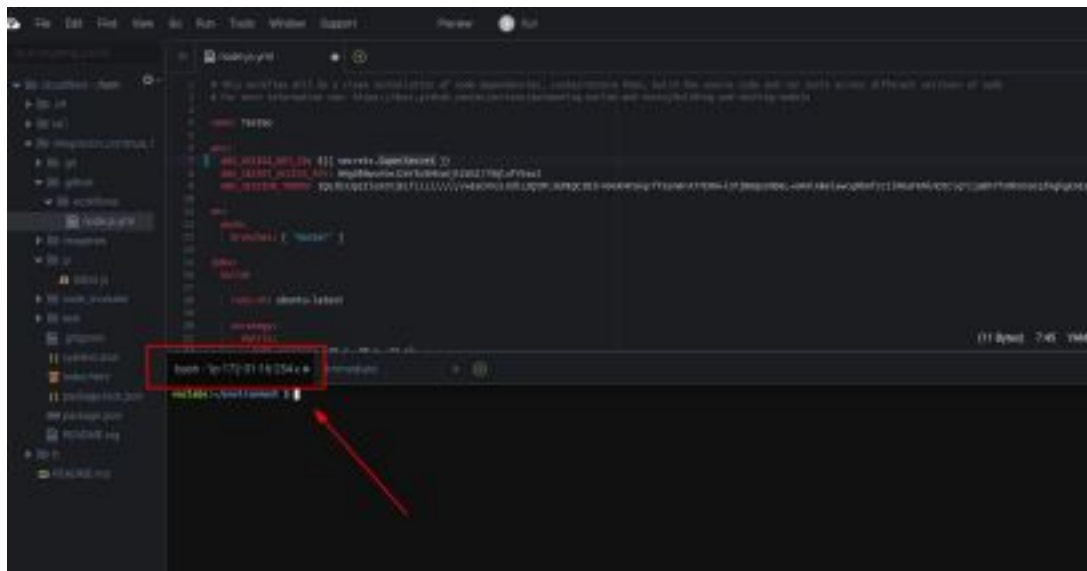
Entorno para prácticas

- Una vez arrancado, conviene ampliar el disco de la instancia de 10Gb a 30Gb. Para ello, ve al servicio EBS, localiza el volumen asociado a la instancia de Cloud9 y modifica su tamaño.
- Una vez hecho eso, te aparecerá un recordatorio de que hay que extender el sistema de ficheros, con un enlace con instrucciones.
- Suponiendo que tengas una instancia de la familia t3, sigue las instrucciones de que corresponden al tipo de hipervisor Nitro (las que vienen por defecto)



Entorno para prácticas

- En el terminal de Cloud9 puedes lanzar los comandos para expandir el sistema de ficheros.



Entorno para prácticas

- Cloud9 incluye un usuario gestionado que despliega sus propias credenciales, asociadas al usuario federado de la consola del Learner Lab.
- En ocasiones, si surgen problemas de permisos, es conveniente asociar el rol “LabRole” a la instancia y desactivar el usuario gestionado.
- De esta manera, los comandos de la CLI se lanzarán con el rol “LabRole”.

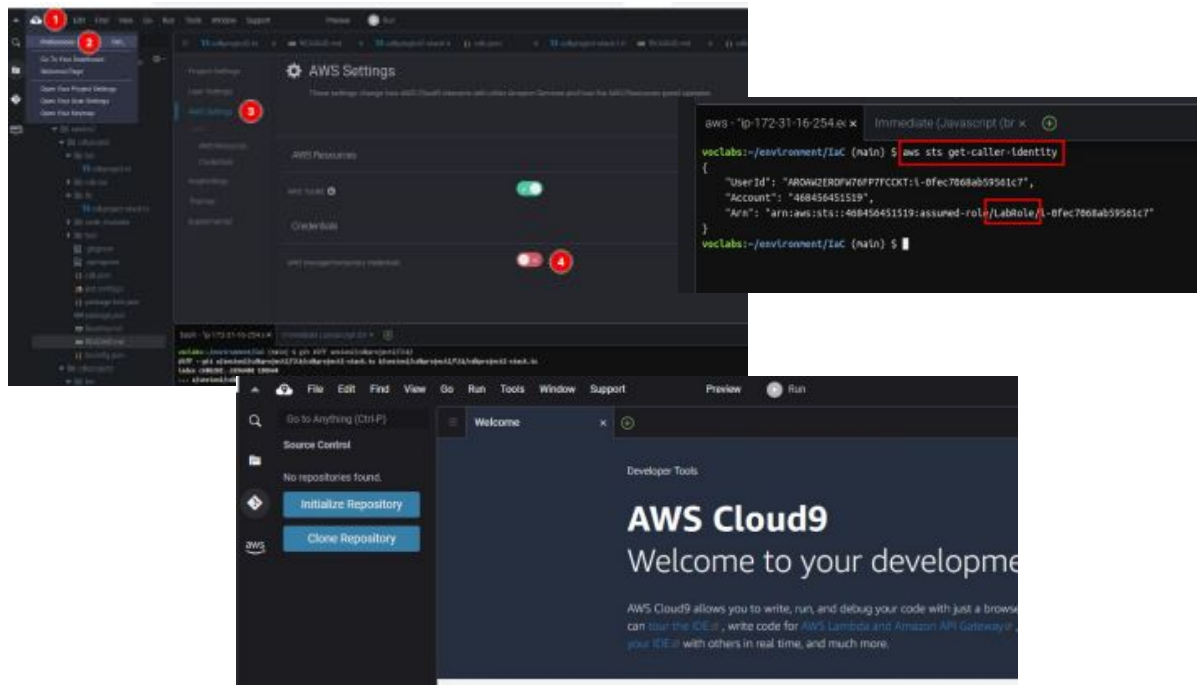


Entorno para prácticas

Desactiva las credenciales gestionadas y asegúrate de esta en LabRole en el terminal.

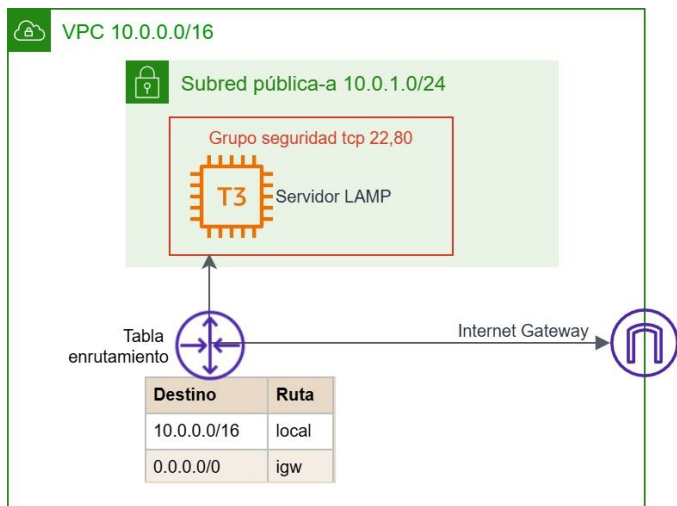
Ahora dentro del entorno de Cloud9 ya puedes trabajar según las instrucciones del profesor, clonando el repositorio git de trabajo para los casos prácticos.

José Luis Gil Gil
Pedro Prieto Alarcón



Objetivos de práctica inicial

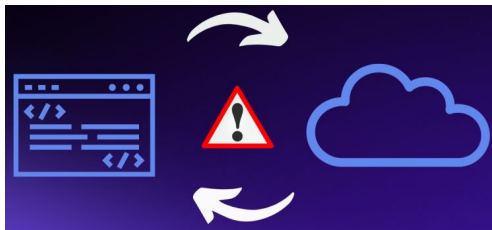
Se trata de comparar herramientas mediante un despliegue de IaC tipo sencillo:



En términos de AWS, se desplegará un VPC con subred pública y una instancia Amazon Linux con stack LAMP instalado mediante user-data y un grupo de seguridad con Inbound abierto sólo a HTTP y SSH.

Objetivos de práctica inicial

Luego se añadirá manualmente una regla en el grupo de seguridad que **permita cualquier tráfico, generando un *drift* entre el código y la infraestructura desplegada, una inconsistencia.**



Veremos los **mecanismos para detectar ese *drift* y revertirlo al estado original correcto** (reconciliación del código y el estado de la infraestructura).

Ejemplo inicial de AWS CloudFormation

Una plantilla YAML de CloudFormation en el repositorio clonado, **generada con AWS Console-to-Code** y ligeramente corregida para ganar sencillez.

Hay instrucciones para despliegue **con la consola web o con la CLI en el IDE Cloud.**

Veremos que CloudFormation es útil para detectar el *drift* detalladamente, pero **no provee de un mecanismo de reconciliación al estado original.**

Ejemplo inicial de Terraform (AWS)

Se trata de un **despliegue similar al anterior** pero realizado con script Terraform.

Necesitaremos primero **preparar el entorno Terraform para AWS dentro del IDE Cloud** con la instalación de las aplicaciones necesarias, las credenciales de AWS las coge automáticamente al estar ya generadas..

Generaremos una regla que abra todos los puertos y veremos cómo podemos **invocar a Terraform para detectar el *drift* y ahora sí que existe un mecanismo de reconciliación** entre el código y el estado de la infraestructura.

Ejemplo inicial de Terraform (Azure)

Ahora realizaremos el mismo despliegue en Azure, adaptado a la forma de hacer las cosas en ese proveedor, que posee recursos equivalentes: virtual network, máquina virtual Ubuntu, Network Security Group asociado a la interfaz de red, IP pública, etcétera

Igualmente deberemos **preparar el entorno de Terraform para Azure** (considerablemente más pesado), y **la autenticación y gestión de las credenciales** será también bastante más compleja: tomaremos la práctica habitual y sencilla de usar variables de entorno.

Igualmente desde el portal de Azure añadiremos una regla que abra todos los puertos y veremos cómo podemos **invocar a Terraform para detectar el *drift* y que de la misma forma se puede hacer reconciliación** entre el código y el estado de la infraestructura.

Como el código es algo más complejo, **se añade un ejemplo modular** en vez de archivo único.

Ejemplo inicial de AWS SAM (API Serverless)

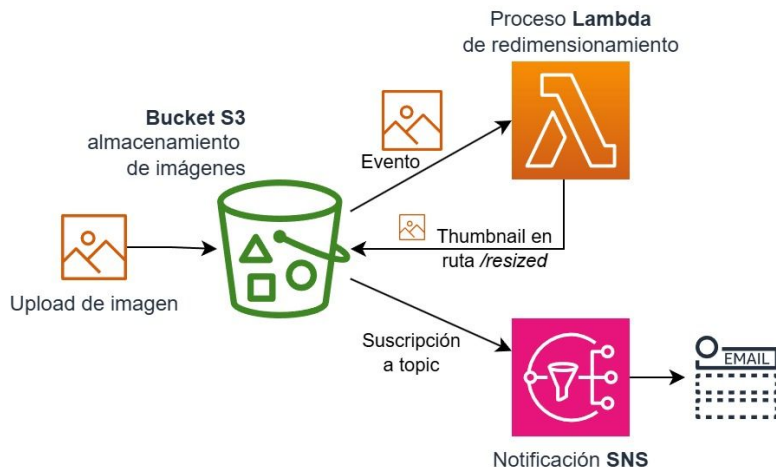
Los despliegues con SAM serán diferentes, dado que es una herramienta orientada a *serverless*: en este caso **es el blueprint de una API sencilla con back en *Lambda***, a la que añadiremos **el front en S3** (fuera de SAM).

Para poder conectar y utilizar el front deberemos **configurar algunos aspectos en el back (para CORS) y en el acceso al bucket S3**.

El código generado por SAM es una **plantilla de CloudFormation**, por lo que las características de **detección de drift y no reconciliación automática**, persisten.

Ejemplo inicial de AWS SAM (Resizer)

En este caso se desplegará una aplicación propia (no blueprint) que **dispara un código *Lambda* de reducción cuando se sube cualquier imagen a un bucket S3**:



Hace uso de ***Lambda*** para hacer ese **procesamiento**, que deja el resultado en el mismo bucket en otra ruta, y de ***SNS*** para **notificar el resultado del proceso por email**.

Ejemplo inicial de AWS CDK (legacy)

Otra vez **volvemos al despliegue de infraestructura no *serverless***, como en CloudFormation y Terraform, pero utilizando la potencia de CDK.

Aunque en último término el código se traduce a una plantilla de CloudFormation, **el lenguaje de alto nivel facilita el desarrollo complejo**.

De nuevo **generaremos un drift** añadiendo una regla que permita todo el tráfico de entrada: se detecta igual y **CDK tampoco tiene un mecanismo de reconciliación** entre infraestructura y código (al final del día, es CloudFormation).

Limitación de AWS Academy: **usaremos el modo *legacy* (sin bootstrap) y no utilizaremos clases de alto nivel** como “Vpc” o “Subnet” **para evitar *lookups***.

Ejemplo inicial de AWS CDK (con bootstrap)

AWS Academy no permite la creación de roles, así que como hemos comentado no es posible realizar el [bootstrapping del proyecto](#) de forma estándar.

Sin embargo, sí se puede **configurar el *bootstrapping* de manera personalizada** para utilizar el rol LabRole proporcionado en el laboratorio de AWS Academy. Para ello, hemos dejado disponible una segunda versión que sigue las detalladas instrucciones indicadas [en este repositorio](#).

No obstante, y aunque el proyecto se puede desplegar, tras introducir la modificación de siempre (regla de apertura de tráfico de entrada) que genera un *drift*, de nuevo **podemos ver que no existe un mecanismo para a cabo la reconciliación entre código e infraestructura**.

Ejemplo inicial de Pulumi

En este caso, instalaremos el entorno dentro del IDE Cloud y le indicaremos que debe trabajar en local (no en *Pulumi Cloud*).

Creamos **un stack nuevo y tras revisar el código generará la infraestructura**. Veremos que se ha creado (al igual que Terraform, **sin generar un código específico CloudFormation por debajo**) y añadiremos la regla de tráfico para generar *drift*.

A diferencia de Terraform o Cloudformation, **el *drift* NO se puede detectar y tampoco tiene mecanismos de reconciliación entre código e infraestructura**.

Ejemplo inicial de CDK for Terraform - CDKTF

En este caso, para poder visualizar la potencia de CDKTF frente a Terraform clásico, **proponemos el estudio (no despliegue real) de IaC para cuatro VPCs similares repartidos en dos regiones (*us-east-1* y *us-west-2*).**

Al poder utilizar bucles y programación de alto nivel, que en último término genera Terraform (y no CloudFormation), **CDKTF reduce drásticamente la repetición de código y es más mantenible que Terraform clásico** en un fichero .tf monolítico.

No obstante, **Terraform con estructura moderna modular** también puede usar bucles, se proporciona también ese código y para este caso, no demasiado complejo, **también podría ser bastante óptimo.**